第二章 知识发现过程与应用结构

内容提要

- 知识发现的基本过程
- 数据库中的知识发现处理过程模型
- 知识发现软件或工具的发展
- 知识发现项目的过程化管理
- 数据挖掘语言介绍

如何学习本章内容

- 在上一章我们指出，数据挖掘有广义和狭义两种理解。为了避免混淆，本章宁愿使用知识发现而把数据挖掘限制在上面所描述的狭义概念上。
- 虽然不同企业会有不同的业务逻辑，解决问题的具体方法有所差异，但是它们进行知识发现的目的和基本思路是一致的。因此，本章首先对知识发现的基本过程进行分析，旨在使读者从总体上掌握知识发现的基本步骤和技术。然后对目前比较流行的KDD过程处理模型进行剖析，使读者了解KDD系统的应用体系结构。通过对KDD系统的基本技术环境和主要部件功能分析，使读者对KDD系统的体系结构有一个更深入的了解。在此基础上对KDD软件和工具进行归纳、举例和分析，帮助读者在实际应用中学会选择和使用相应的软件和工具。
- 本章也对KDD系统项目的过程化管理、交互式数据挖掘过程以及通用的KDD原型系统进行讨论，使读者从软件项目管理角度来更好地理解KDD过程。最后对数据挖掘语言的类型和特点进行介绍。

知识发现是一个系统化的工作

- 从源数据中发现有用知识是一个系统化的工作。首先必须对可以利用的源数据进行分析，确定合适的挖掘目标。然后才能着手系统的设计和开发。
- 完成从大型源数据中发现有价值知识的过程可以简单地概括为：首先从数据源中抽取感兴趣的数据，并把它组织成适合挖掘的数据组织形式；然后，调用相应的算法生成所需的知识；最后对生成的知识模式进行评估，并把有价值的知识集成到企业的智能系统中。
- 一般地说，KDD是一个多步骤的处理过程，一般分为问题定义、数据抽取、数据预处理、数据挖掘以及模式评估等基本阶段。

问题定义阶段的功能

- KDD是为了在大量数据中发现有用的令人感兴趣的信息，因此发现何种知识就成为整个过程中第一个也是最重要的一个阶段。
- 在问题定义过程中，数据挖掘人员必须和领域专家以及最终用户紧密协作
  - 一方面了解相关领域的有关情况，熟悉背景知识，弄清用户要求，确定挖掘的目标等要求；
  - 另一方面通过对各种学习算法的对比进而确定可用的学习算法。后续的学习算法选择和数据集准备都是在此基础上进行的。

数据抽取与集成

- 数据抽取与集成是知识发现的关键性工作。早期的数据抽取是依靠手工编程来实现的，现在可以通过高效的抽取工具来实现。即使是使用抽取工具，数据抽取和和装载仍然是一件很艰苦的工作。
- 源数据库的选取以及从中抽取数据的原则和具体规则必须依据系统的任务来界定。主要任务是设计存储新数据的结构和准确定义它与源数据的转换和装载机制，以便正确地从每个数据源中抽取所需的数据。这些结构和转换信息应该作为元数据（Metadata）被存储起来。
- 来自不同源的数据可能有模式定义上的差异，也可能存在因数据冗余而无法确定有效数据的情形。此外，还要考虑数据库系统本身可能存在不兼容的情况

数据清洗与预处理

- 在开始一个知识发现项目之前必须清晰地定义挖掘目标。虽然挖掘的最后结果是不可预测的，但是要解决或探索的问题应该是可预见的。盲目性地挖掘是没有任何意义的。如果所集成的数据不正确，数据挖掘算法输出的结果也必然不正确，这样形成的决策支持是不可靠的。因此，要提高挖掘结果的准确率，数据预处理是不可忽视的一步。
- 对数据进行预处理，一般需要对源数据进行再加工，检查数据的完整性及数据的一致性，对其中的噪音数据进行平滑，对丢失的数据进行填补，消除“脏”数据，消除重复记录等。
- 数据清洗是指去除或修补源数据中的不完整、不一致、含噪音的数据。在源数据中，可能由于疏忽、懒惰、甚至为了保密使系统设计人员无法得到某些数据项的数据。假如这个数据项正是知识发现系统所关心的，那么这类不完整的数据就需要修补。
- 常见的不完整数据的修补办法有：
  - 使用一个全局值来填充（如“unknown”、估计的最大数或最小数）。
  - 统计该属性的所有非空值，并用平均值来填充空缺项。
  - 只使用同类对象的属性平均值填充。
  - 利用回归或工具预测最可能的值，并用它来填充。
- 数据不一致可能是由于源数据库中对同样属性所使用的数据类型、度量单位等不同而导致的。因此需要定义它们的转换规则，并在挖掘前统一成一个形式。噪音数据是指那些明显不符合逻辑的偏差数据（如某雇员200岁），这样的数据往往影响挖掘结果的正确性。

数据的选择与整理

- 没有高质量的数据就不可能有高质量的挖掘结果。为了得到一个高质量的适合挖掘的数据子集，一方面需要通过数据清洗来消除干扰性数据，另一方面也需要针对挖掘目标进行数据选择。数据选择的目的是辨别出需要分析的数据集合，缩小处理范围，提高数据采掘的质量。数据选择可以使后面的数据挖掘工作聚焦到和挖掘任务相关的数据子集中。不仅提高了挖掘效率，而且也保证了挖掘的准确性。
- 利用数据变换或规约等技术可以将数据整理成适合进一步挖掘的数据格式。数据变换可以根据需要构造出新的属性以帮助理解分析数据的特点，或者将数据规范化，使之落在一个特定的数据区间中。数据归约则是在尽可能保证数据完整性的基础上，将数据以其他方式进行表示，以减少数据存储空间，使挖掘过程更有效。常用的归约策略有：数据立方体聚集、维归约、数据压缩、数值压缩和离散化等。

数据挖掘阶段的功能

- 运用选定的数据挖掘算法，从数据中提取出用户所需要的知识，这些知识可以用一种特定的方式表示。
- 选择数据挖掘算法有两个考虑因素：
  - 一是不同的数据有不同的特点，因此需要用与之相关的算法来挖掘；
  - 二是用户或实际运行系统的要求，有的用户可能希望获取描述型的（Descriptive）、容易理解的知识（采用规则表示的挖掘方法显然要好于神经网络之类的方法），而有的用户只是希望获取预测准确度尽可能高的预测型（Predictive）知识，并不在意获取的知识是否易于理解。

知识评估阶段的功能

- 数据挖掘阶段发现出来的模式，经过评估，可能存在冗余或无关的模式，这时需要将其剔除；也有可能模式不满足用户要求，这时则需要整个发现过程回退到前续阶段，如重新选取数据、采用新的数据变换方法、设定新的参数值，甚至换一种算法等等。
- KDD由于最终是面向人类用户的，因此可能要对发现的模式进行可视化，或者把结果转换为用户易懂的另一种表示。所以知识评估阶段是KDD一个重要的必不可少的阶段，它不仅担负着将KDD系统发现的知识以用户能了解的方式呈现，而且根据需要进行知识评价，如果和用户的挖掘目标不一致就需要返回前面相应的步骤进行螺旋式处理以最终获得可用的知识。

知识发现需要多方合作

- 实施这样的项目不仅需要充足的资金，而且需要有良好的技术和人员储备。在整个的知识发现过程中，需要有不同专长的技术人员支持。
  - 业务分析人员：要求精通业务，能够解释业务对象，并根据各业务对象确定出用于数据定义和挖掘算法的业务需求。
  - 数据分析人员：精通数据分析技术，并对统计学有较熟练的掌握，有能力把业务需求转化为知识发现的各步操作，并为每步操作选择合适的模型或工具。
  - 数据管理人员：精通数据管理技术，并负责从数据库或数据仓库中收集数据。

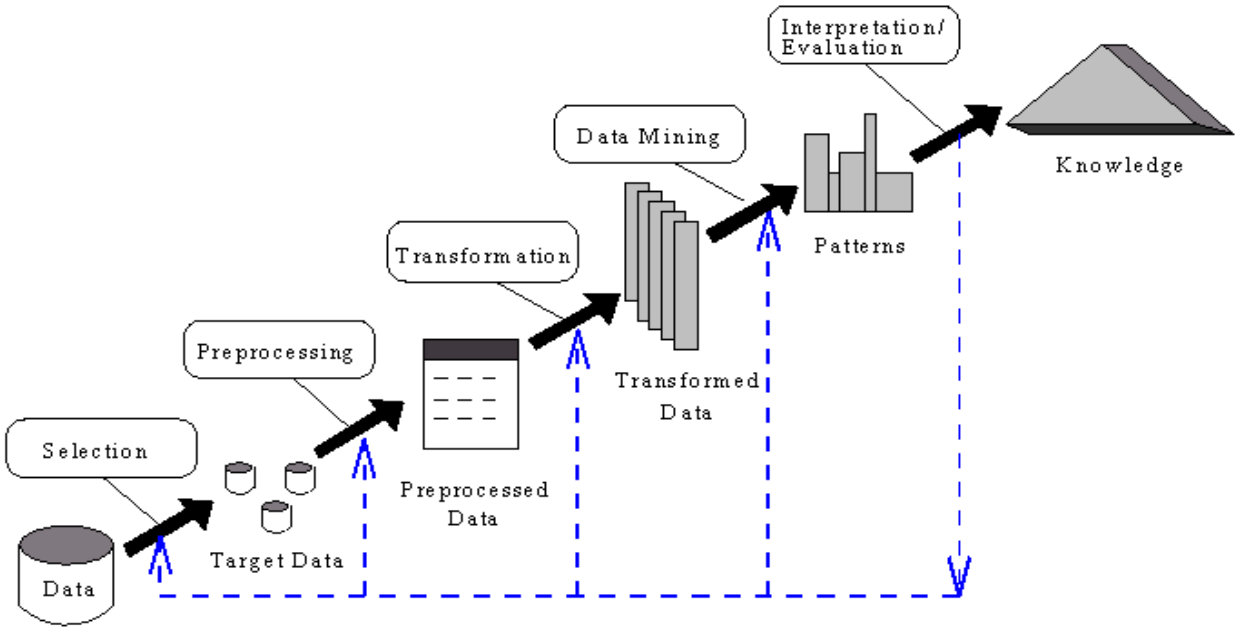
第二章 知识发现过程与应用结构

内容提要

- 知识发现的基本过程
- 数据库中的知识发现处理过程模型
- 知识发现软件或工具的发展
- 知识发现项目的过程化管理
- 数据挖掘语言介绍

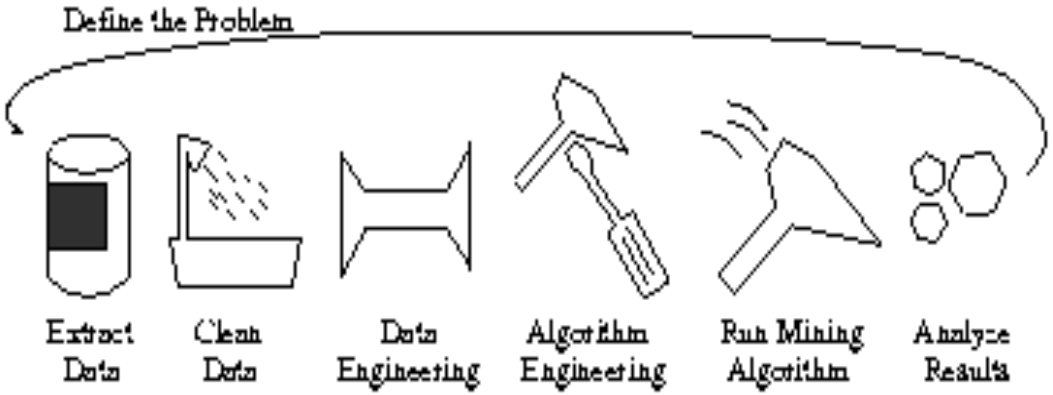
阶梯处理过程模型

- 多阶段流水处理模型：



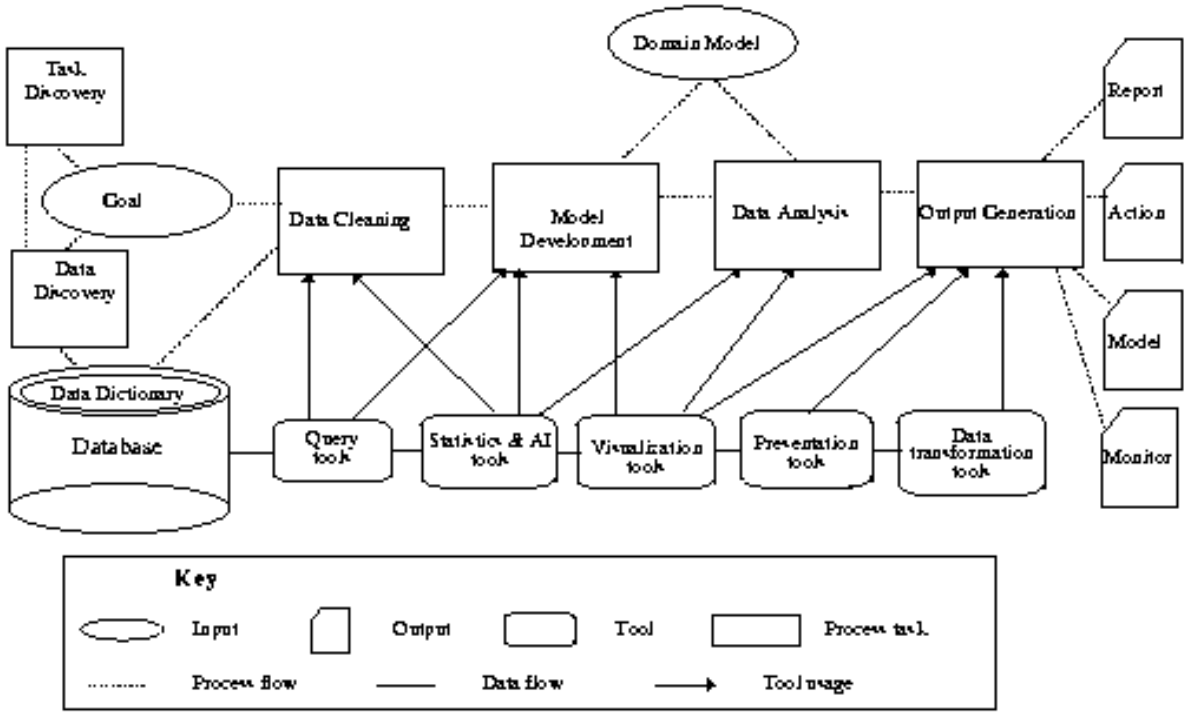
螺旋处理过程模型

- 它强调领域专家参与的重要性，并以问题的定义为中心循环评测挖掘的结果。当结果不令人满意时，就需要重新定义问题，开始新的处理循环。每次循环都使问题更清晰，结果更准确，因此是一个螺旋式上升过程。



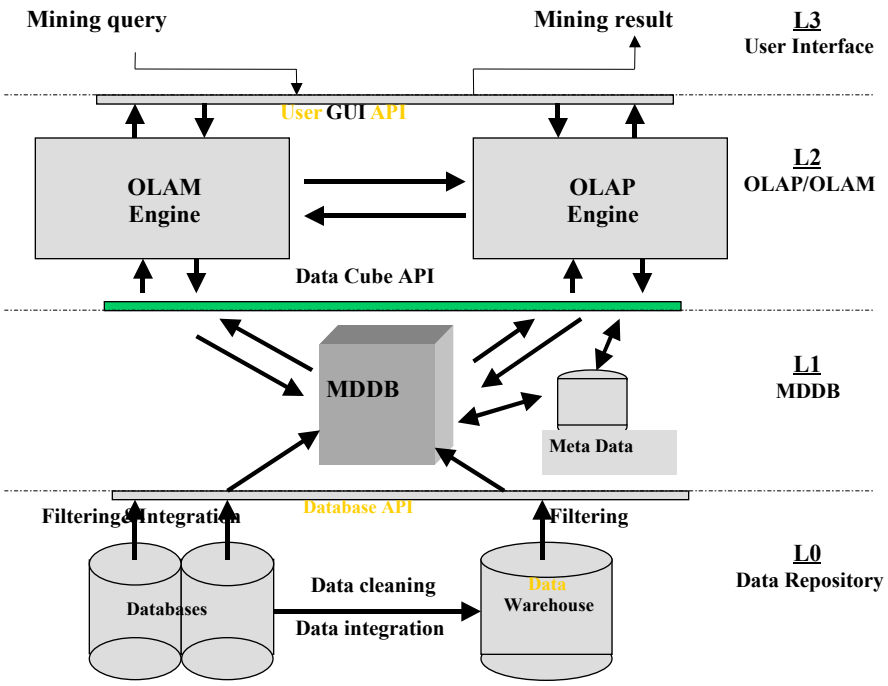
以用户为中心的处理模型

- Brachman和Anand从用户的角度对KDD处理过程进行了分析。他们认为数据库中的知识发现应该更着重于对用户进行知识发现的整个过程的支持，而不是仅仅限于在数据挖掘的一个阶段上。该模型强调对用户与数据库的交互的支持。



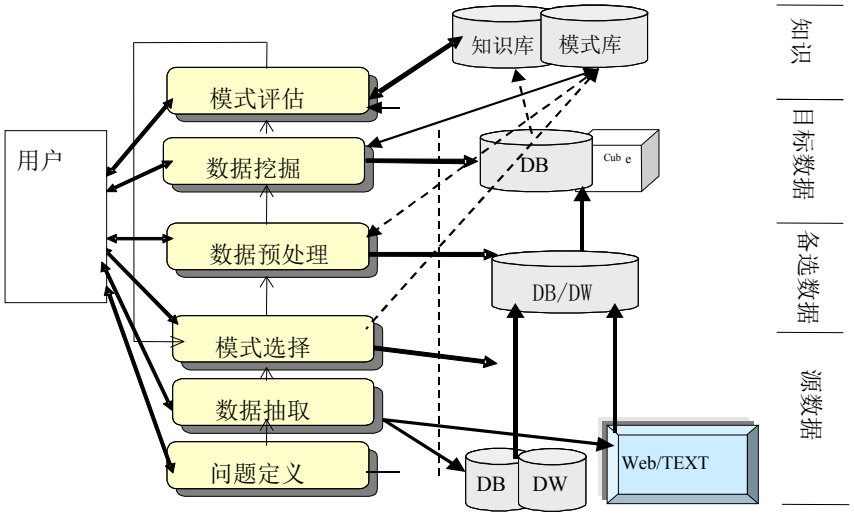
联机KDD模型

- 实现联机交互式KDD需要可视化技术支撑。这种可视化需要从数据挖掘过程可视化、数据可视化、模型可视化和算法可视觉化等方面来理解。
- OLAM（On Line Analytical Mining：联机分析挖掘）的概念是OLAP的发展。



支持多数据源多知识模式的KDD处理模型

- 数据与方法相对独立。数据不是针对某一特定知识模式，而是针对某一类问题来抽取。经过预处理后，这些数据对于某些挖掘算法来说可能存在属性冗余、与目标无关等问题，因此在后面的阶段再进行相关的数据清洗和选择工作，这样使得解决同一类问题的不同算法可以在统一的KDD平台上完成。



第二章 知识发现过程与应用结构

内容提要

- 知识发现的基本过程
- 数据库中的知识发现处理过程模型
- 知识发现软件或工具的发展
- 知识发现项目的过程化管理
- 数据挖掘语言介绍

知识发现软件的发展

- 虽然市场上已经有许多所谓的知识发现系统或工具，但是，这些工具只能用来辅助技术人员进行设计和开发，而且知识发现软件本身也正处于发展阶段，仍然存在各种各样需要解决的问题。
- 粗略地说，知识发现软件或工具的发展经历了独立的知识发现软件、横向的知识发现工具集和纵向的知识发现解决方案三个主要阶段，其中后面两种反映了目前知识发现软件的两个主要发展方向。

独立的知识发现软件

- 独立的知识发现软件出现在数据挖掘和知识发现技术研究的早期。当研究人员开发出一种新型的数据挖掘算法后，就在此基础上形成软件原型。这些原型系统经过完善被尝试使用。
- 这类软件要求用户必须对具体的数据挖掘技术和算法有相当的了解，还要手工负责大量的数据预处理工作。

横向的知识发现工具

- 集成化的知识发现辅助工具集，属于通用辅助工具范畴，可以帮助用户快速完成知识发现的不同阶段处理工作。
- 一些有代表性的原型系统或工具介绍。

名称	研究机构或公司	主要特点
DBMiner[1]	Simon Fraser	以OLAM引擎为核心的联机挖掘原型系统；包含多特征/序列/关联等多模式。
Quest[75]	IBM Almaden	面向大数据集的多模式（关联规则/分类等）挖掘工具。
IBM Intelligent Miner[76]	IBM	包含多种技术（神经网络/统计分析/聚类等）的辅助挖掘工具集。
Darwin[76]	Thinking Machines	基于神经网络的辅助挖掘工具。
ReMind	Cognitive System	基于实例推理和归纳逻辑的辅助挖掘工具。

纵向的知识发现解决方案

- 随着横向的数据挖掘工具集的使用日渐广泛，人们也发现这类工具只有精通数据挖掘算法的专家才能熟练使用。如果对数据挖掘和知识发现技术及其算法不了解，就难以开发出好的应用。
- 纵向的数据挖掘解决方案的核心是针对特定的应用提供完整的数据挖掘和知识发现解决方案。由于和具体的商业逻辑相结合，因此，数据挖掘技术专门为了解决某些特定的问题被使用，成为企业应用系统中一部分。
- 许多厂商或研究机构可以提供纵向数据挖掘的解决方案。因此，数据挖掘技术在最近几年开始在一些领域得到应用。例如，证券系统的趋势预测、银行和电信行业的欺诈行为检测、在CRM中的应用、在基因分析系统中用于DNA识别等。
- 特定领域的数据挖掘工具针对性比较强，只能用于一种应用；也正因为针对性强，往往采用特殊的算法，可以处理特殊的数据，实现特殊的目的，发现的知识可靠度也比较高。

第二章 知识发现过程与应用结构

内容提要

- 知识发现的基本过程
- 数据库中的知识发现处理过程模型
- 知识发现软件或工具的发展
- 知识发现项目的过程化管理
- 数据挖掘语言介绍

知识发现项目的过程化管理问题

- 开发一个数据挖掘和知识发现项目需要各方面协同合作而且极易出现问题，因此它的质量管理问题的讨论是重要而困难的。
- 近几年，有一些针对数据挖掘和知识发现项目的过程化管理所开展的工作，其中一个典型的模型三被称作强度挖掘（Intension Mining）的I-MIN过程模型。
- I-MIN过程模型把KDD过程分成IM1、IM2、…、IM6等步骤处理，在每个步骤里，集中讨论几个问题，并按一定的质量标准来控制项目的实施。

IM1的任务与目的

- 它是KDD项目的计划阶段，需要确定企业的挖掘目标，选择知识发现模式，编译知识发现模式得到的元数据。其目的是将企业的挖掘目标嵌入到对应的知识模式中。
- 对数据挖掘研究人员来说，往往把主要精力用在改进现有算法和探索新算法上。但是在真正调用挖掘算法之前，必须对企业的决策机制和流程进行充分调研，理解企业急需解决的问题。需要准确地确定挖掘目标和可交付系统的指标等。

IM2的任务与目标

- 它是KDD的预处理阶段，可以用IM2a、IM2b、IM2c等分别对应于数据清洗、数据选择和数据转换等阶段。其目的是生成高质量的目标数据。
- 知识发现项目的数据预处理是一个费时费力的工作。事实上，数据挖掘的成功与否，数据预处理起到了至关重要的作用。只有好的预处理，才能避免GIGO（垃圾进垃圾出）的现象发生。

IM3的任务与目标

- 它是KDD的挖掘准备阶段，数据挖掘工程师进行挖掘实验，反复测试和验证模型的有效性。其目的是通过实验和训练得到浓缩知识(Knowledge Concentrate)，为最终用户提供可使用的模型。

IM4的任务与目标

- 它是KDD的数据挖掘阶段，用户通过指定数据挖掘算法得到对应的知识。

IM5的任务与目标

- 它是KDD的知识表示阶段，按指定要求形成规格化的知识。

IM6的任务与目标

- 它是KDD的知识解释与使用阶段，其目的是根据用户要求直观地输出知识或集成到企业的知识库中。

第二章 知识发现过程与应用结构

内容提要

- 知识发现的基本过程
- 数据库中的知识发现处理过程模型
- 知识发现软件或工具的发展
- 知识发现项目的过程化管理
- 数据挖掘语言介绍

数据挖掘语言概述

- 设计理想的数据挖掘语言是一个巨大的挑战。这是因为数据挖掘覆盖的任务宽、包含知识形式广（如数据特征化、关联规则、数据分类、聚集等等）。每个任务都有不同的需求，每种知识表示形式都有不同内涵。一个有效的数据挖掘语言设计需要对各种不同的数据挖掘任务的能力、约束以及运行机制有深入地理解。
- 众所周知，关系查询语言的标准化，发生在关系型数据库开发的早期阶段。经过不懈的努力，以SQL为代表的关系型数据库查询语言的标准化被成功解决。同样，一个好的数据挖掘语言可以有助于数据挖掘系统平台的标准化进程，甚至可以象HTML推动Internet的发展一样，推动数据挖掘行业的开发和发展。
- 数据挖掘语言的发展大致经历了两个阶段：第一个阶段是研究单位和公司自行研究和开发阶段；第二阶段是研究单位和公司组成联盟，研制和开发数据挖掘语言标准化的阶段。

数据挖掘语言概述

- **根据功能和侧重点不同，数据挖掘语言可以分为三种类型：**
  - 数据挖掘查询语言：希望以一种像SQL这样的数据库查询语言完成数据挖掘的任务。
  - 数据挖掘建模语言：对数据挖掘模型进行描述和定义的语言，设计一种标准的数据挖掘建模语言，使得数据挖掘系统在模型定义和描述方面有标准可以遵循。
  - 通用数据挖掘语言：通用数据挖掘语言合并了上述两种语言的特点，既具有定义模型的功能，又能作为查询语言与数据挖掘系统通信，进行交互式挖掘。通用数据挖掘语言的标准化是目前解决数据挖掘行业出现问题的颇具吸引力的研究方向。

数据挖掘查询语言

- J. W. Han等开发的数据挖掘系统DBMiner中数据挖掘查询语言DMQL（Data Mining Query Language）是这类挖掘语言的典型代表。数据挖掘查询语言DMQL由数据挖掘原语组成，数据挖掘原语用来定义一个数据挖掘任务。用户使用数据挖掘原语与数据挖掘系统通信，使得知识发现更有效。
- 这些原语有以下几个种类：
  - 数据库部分以及用户感兴趣的数据集（包括感兴趣的数据库属性或数据仓库的维度）；
  - 挖掘知识的种类；在指导挖掘过程中有用的背景知识；
  - 模式估值的兴趣度测量；挖掘出的知识如何可视化表示。
- 数据挖掘查询的基本单位是数据挖掘任务，通过数据挖掘查询语言，数据挖掘任务可以通过查询的形式输入到数据挖掘系统中。一个数据挖掘查询由五种基本的数据挖掘原语定义。

数据挖掘建模语言

- 数据挖掘建模语言是对数据挖掘模型进行描述和定义的语言。
- 预言模型标记语言”（Predictive Model Markup Language，PMML）被一个称作数据挖掘协会（The Data Mining Group，DMG）的组织开发。PMML是一种基于XML的语言，用来定义预言模型。PMML允许应用程序和联机分析处理（OLAP）工具能从数据挖掘系统获得模型，而不用独自开发数据挖掘模块。
- PMML的模型定义由以下几部分组成：
  - 头文件（Header）；
  - 数据模式（Data Schema）；
  - 数据挖掘模式（Data Mining Schema）；
  - 预言模型模式（Predictive Model Schema）；
  - 预言模型定义（Definitions for Predictive Models）；
  - 全体模型定义（Definitions for Ensembles of Models）；
  - 选择和联合模型和全体模型的规则（Rules for Selecting and Combining Models and Ensembles of Models）；
  - 异常处理的规则（Rules for Exception Handling）

通用数据挖掘语言

- 通用数据挖掘语言合并了上述两种语言的特点，既具有定义模型的功能，又能作为查询语言与数据挖掘系统通信，进行交互式挖掘。通用数据挖掘语言的标准化是目前解决数据挖掘行业出现问题的颇具吸引力的研究方向。
- 2000年3月，微软公司推出了一个数据挖掘语言，称作OLE DB for Data Mining（DM），是通用数据挖掘语言中最具代表性的尝试。微软此举的目的是为数据挖掘提供行业标准。只要符合这个标准，都能容易地嵌入应用程序中。
- OLE DB for DM支持多种流行的数据挖掘算法。使用OLE DB for DM，数据挖掘应用能够通过OLE DB生产者接进任何表格式的数据源。

DMQL挖掘查询语言介绍

- **DMQL语言的顶层语法**

{DMQL}::=<DMQL_Statement>；{<DMQL_Statement>}
<DMQL_Statement>::=<Data_Mining_Statement>
|<Concept_Hierarchy_Definition_Statement>
|<Visualization_and_Presentation>

- **数据挖掘声明（Data_Mining_Statement）语句相关项说明**

<Data_Mining_Statement>::=use database <database_name>
| use data warehouse <data_warehouse_name>
{use hierarchy <hierarch_name> for
<attribute_or_dimension>}
from <relation(s)/cube(s)> [where <condition>]
in relevance to <attribute_or_dimension_list>
[order by <order_list>]
[group by <grouping_list>]
[having <condition>]

- 例子：

use database AllElecronics_db
in relevance to I.name,I.price,C.income,C.age
from customer C, item I, purchases P, items_sold S
where I.item_ID=S.item_ID and S.trans_ID=P.trans_ID and P.cust_ID=C.cust_ID and
C.country="Canada"
group by P.date；

DMQL挖掘查询语言介绍(续)

- **挖掘知识指定（Mine_Knowledge_Specification）语句相关项说明**

<Mine_Knowledge_Specification>::=<Mine_Char>|<Mine_Discr>|<Mine_Assoc>|<Mine_Class>

<Mine_Char> ::= mine characteristics [as <pattern_name>] analyze <measure(s)>

<Mine_Discr> ::= mine comparison [as <pattern_name>]
for <target_class> where <target_condition>
{versus <contrast_class_i> where
<contrast_condition_i>}
analyze <measure(s)>

<Mine_Assoc> ::= mine associations [as <pattern_name>]
[matching <metapattern>]

<Mine_Class> ::=mine classification [as <pattern_name>]
analyze <classifying_attribute_or_dimension>

DMQL挖掘查询语言介绍(续)

- 概念分层声明（Concept_Hierarchy_Definition_Statement）相关项说明

<Concept_Hierarchy_Definition_Statement> ::=define hierarchy
<hierarchy_name> [for <attribute_or_dimension>]
on <relation_or_cube_or_hierarchy>
as <hierarchy_description>
[where <condition>]

- 例子：

define hierarchy age_hierarchy *for* age *on* customer *as*
level1: {young, middle_aged, senior} < level0:*all*
level2: {20, ..., 39} < level1: *young*
level2: {40, ..., 59} < level1: *middle_aged*
level2: {60, ..., 89} < level1: *senior*;
define hierarchy profit_margin_hierarchy *on* item *as*
level1: low_profit_margin < level_0: *all*
if (price - cost)< $50
level1: medium-profit_margin < level_0: *all*
if ((price - cost) > $50) and ((price - cost) <= $250))
level1: high_profit_margin < level_0: *all*
if (price - cost) > $250;

DMQL挖掘查询语言介绍(续)

- **模式表示和可视化说明的语法**

<Visualization_and_Presention> ::=display as
<result_form> | {<Multilevel_Manipulation>};
<Multilevel_Manipulation> ::= roll up on <attribute_or_dimension>
| drill down on <attribute_or_dimension>
| add <attribute_or_dimension>
| drop <attribute_or_dimension>;

其中<result_form>可以是规则、表、交叉表、饼图或条图、判定树、立方体、曲线或曲面等

第二章 知识发现过程与应用结构

内容提要

- 知识发现的基本过程
- 数据库中的知识发现处理过程模型
- 知识发现软件或工具的发展
- 知识发现项目的过程化管理
- 数据挖掘语言介绍

Thank you !!!